Project Overview

Multirotor drones have seen increased utilization for present day innovations; applications such as delivery, racing, or flying near structures, require accurate path following. For smaller sized drones, path following becomes increasingly difficult in turbulent wind environments; as multirotor drones maintain stability by constantly adjusting rotor speeds, they require significant control authority. In conditions with strong or gusty wind present, the onboard controller is forced to work harder, their batteries get drained more rapidly, and its ability to maintain stability may be exceeded. This is especially true for smaller craft, as their lightweight nature renders them more sensitive to turbulence.

Therefore, many tools have been developed to simulate autopilot conditions for multirotor drones. A prime example is PX4, an open-source autopilot widely used in research settings to complement real flight testing through a controlled environment and repeatable trials. Running it in software-in-the-loop (SITL) simulation with the Gazebo simulator allows flight code to be tested against simulated physics. Given that PX4 has a stock performance baseline that implements a cascaded position, velocity, attitude and rate controllers with specific tuning defaults, a user that is building atop of PX4, or designing a new controller, can henceforth understand the performance of the baseline and customize their own systems and improve control logic for their own applications.

This leads to the purpose of the project, which is to determine how well PX4's stock default controller tracks a figure-8 (Lemniscate of Gerono) trajectory under gusty wind and differing inertial and mass factors. This is conducted through a SITL Monte-Carlo simulation with Gazebo, implementing randomized gusty wind, and mass and inertia mismatch. The final deliverables are the measured horizontal root-mean-square (RMS) tracking error between the actual and desired figure-8 loop trajectories, complemented by bootstrap confidence intervals. Therefore, the final question can be posed as the following: *how well does PX4's stock controller track a figure-8 trajectory under gusty wind and model mismatch?*

This project will be documented through a series of sections:

- **(A) Simulation Setup:** describes the flight stack and how wind and model mismatch is introduced
- **(B) Experimental Design:** defines the trial sequence, variables, RMS error, and bootstrap method
- **(C) Results Analysis:** bootstrap estimates and figures showing tracking error against metrics
- **(D) Discussion:** interpret the results to answer the research question
- **(E) Limitations:** outlines the constraints of the estimator, wind model, and experimental design
- **(F) Further Expansions:** proposes follow-up work on turbulence models and hardware validation

The full code documentation can be found on GitHub: https://github.com/ItsAerocraft/flightstack-learn.

(A) Simulation Setup

The simulation flight stack is conducted within a WSL2 Ubuntu 24.04 environment and consists of three coupled components, as depicted in **Figure 1**: PX4 (the autopilot flight control software), Gazebo (the physics, sensor and wind simulator), and the MAVSDK-Python scripts (the offboard control script). The control script calculates the target trajectory through a series of equations, and streams position and velocity setpoints to PX4 via MAVLink. Simultaneously, PX4 and Gazebo exchange simulated sensor data and motor commands; simulated time only advances when both these components are ready. This ensures that slow computational speed does not disrupt physical simulation.

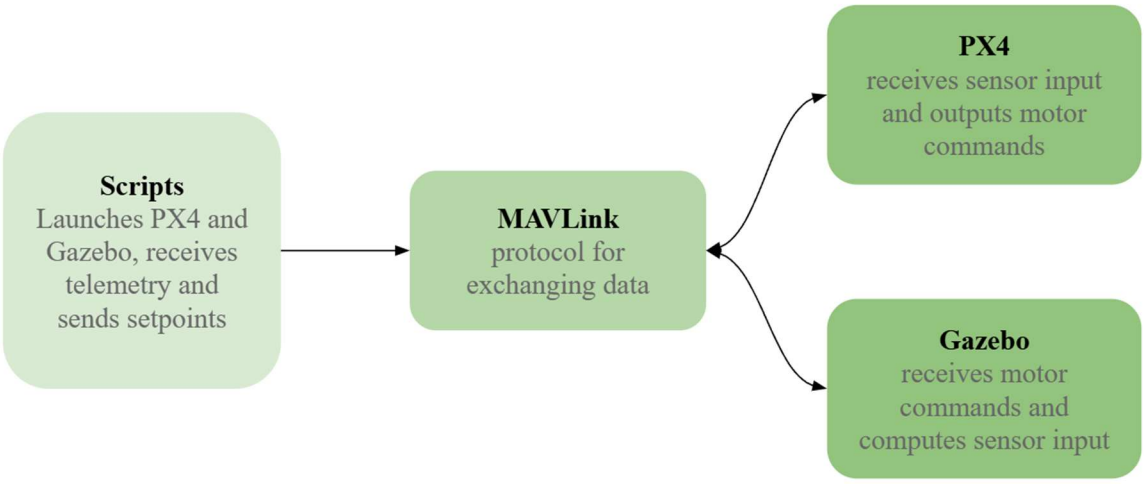


Figure 1 – Communication Pathway Between Components

The simulated vehicle is the standard x500 quadrotor. To accomplish project objectives, Gazebo modifies the vehicle's physical inertial properties (mass and inertia) per trial by up to ±20% to simulate model mismatch. This mechanism tests the robustness of the stock controller against unmodelled payloads or the drone's physical variations, as PX4's internal controller tuning remains fixed at their nominal values.

PX4's stock cascade controller is used for all trials. The Python script operates in PX4's "offboard mode", which controls the vehicle's movement and attitude by streaming position and velocity setpoints at a fixed frequency of 50 Hz, where the velocity setpoint acts as the primary feed-forward term. The actual position of the drone is simultaneously received and derived from PX4's Extended Kalman Filter (EKF) state estimate. A fixed-deadline loop was implemented to propagate time, to account for and count late ticks as a result from the computer processing background tasks; this solved the issue of an original measured 33 Hz streaming frequency. During the testing and script running phase, no ticks were recorded as more than one period late.

Wind is injected through the simulation world SDF file that Gazebo uses to describe the physical simulation environments. Specifically, this is done by modifying Gazebo's WindEffects plugin by parsing the default world file and creating a modified file through the Python ElementTree library.

The basic settings and setup requirements to run the experiment are presented below:

Table 1 – Simulation and Vehicle Settings

Setting	**Value / Description**
PX34 Autopilot Version	v1.18.0-beta1 development build
Gazebo Version	Gazebo Sim 8.15.0 (Harmonic)
MAVSDK Python Version	Lower than version 4
Python Version	Version 3.10 or higher
Host Operating System (OS)	Windows 11 Home, running Ubuntu 24.04.5 LTS on WSL2
Vehicle Model	x500 quadrotor (PX4 airframe 4001)
Offboard Control Setpoint Type	Position + velocity in NED at 50 Hz

The parameters not tabulated but introduced in this section will be discussed in the following section.

(B) Experimental Design

The main experimental comparison between trajectories is with the reference or target trajectory, a Lemniscate of Gerono (figure-8) flown at a constant altitude. The figure-8 was chosen because it combines relatively straight segments, tight turns, and direction changes in both horizontal axes. This would therefore thoroughly test tracking performance along every horizontal direction.

The figure-8 shape is defined by the parametric equations:

$$x_d(t) = A\sin(\omega t), \qquad y_d(t) = \frac{1}{2}A\sin(2\omega t), \qquad z_d(t) = -h \tag{B.1}$$

- x_d, y_d, z_d form the position vector $\langle x_d, y_d, z_d \rangle$ in North-East-Down (NED) coordinates (m), h is the altitude of the drone (m), and t is the time parameter (s), replaced by τ during the smooth ramp
- A is its amplitude (m) of the Lemniscate, and ω is the angular frequency of the Lemniscate

These parametric equations, as mentioned, are in NED coordinates, therefore requires $z_d(t)$ to be encoded with the negative of drone altitude to satisfy the "down" component.

This parametrization therefore yields a peak acceleration a_{peak} of approximately:

$$a_{peak} = 2A\omega^2 \tag{B.2}$$

This peak acceleration is the metric defining how aggressive the maneuver is.

However, this parametrization yields a non-zero velocity at $t = 0$, which corresponds to the hovering drone experiencing a step input if the trajectory started abruptly. To prevent this, rather than following the trajectory in real time t, the drone follows a warped time τ, whose rate $\dot{\tau}$ rises from 0 to 1 over the ramp duration T_{ramp}, during which all performance metrics are excluded:

$$\dot{\tau} = s(u) = 3u^2 - 2u^3, \qquad u = \min\left(\frac{t}{T_{ramp}}, 1\right) \tag{B.3}$$

- s is the playback rate of the trajectory, rising smoothly from 0 to 1
- u is the normalized ramp time, rising from 0 to 1 during the ramp, and remaining at 1 afterwards

This smooth step function is chosen because its derivative is zero at both $u = 0$ and $u = 0$, so the ramp begins and ends without abrupt changes. Applying the chain rule to $x_d(\tau(t))$ therefore gives the scaled position, velocity and acceleration:

$$x_{scaled}(\tau) = x_d(\tau), \qquad v_{scaled}(\tau) = [v_d(\tau)]s, \qquad a_{scaled}(\tau) = [a_d(\tau)]s^2 + [v_d(\tau)]\dot{s} \tag{B.4}$$

- x_{scaled},v_{scaled}, a_{scaled} are the respective analytical position, velocity, and acceleration terms of the reference trajectory

At the start of the ramp, $s = \dot{s} = 0$, so the scaled velocity and acceleration are zero, which matches the hovering state. After the ramp, $s = 1$ and $\dot{s} = 0$, thus the drone returns to the original trajectory with unwarped time. The second acceleration term, $[v_d(\tau)]\dot{s}$, is a purely tangential term that arises only while the playback rate is changing.

Each simulated trial follows a strict sequence and consists of n_{lap} laps. First, PX4 and Gazebo launch headless. Thereafter, the drone arms and takes off to a hover altitude of h meters in calm conditions. Wind is then activated after the drone reaches the target altitude, and a T_{settle} second settling time allows the controller to stabilize under the new wind conditions. Afterwards, the smooth step ramp begins, followed by the n_{lap} figure-8 laps. Finally, the drone lands, and all processes are cleanly terminated.

To evaluate robustness, each trial draws randomized parameters from a seed arising from the Python NumPy library's random.default_rng, which would ensure every trial is exactly reproducible from its seed. These randomized parameters each are of uniform distribution and have a set range.

The primary metric is the horizontal RMS position error σ over the evaluation window (from the end of the ramp-in to the end of the final lap):

$$\sigma = \sqrt{\frac{1}{N}\sum_{i=1}^{N}[(x_i - (x_d)_i)^2 + (y_i - (\mathrm{y_d})_i)^2]}$$

- N is the number of logged samples in the evaluation window, and i is the sample index
- x_i, y_i correspond to PX4's estimated horizontal position at sample i
- $(x_d)_i, (y_d)_i$ correspond to the commanded horizontal position at sample i

The metric is restricted to the horizontal plane because wind and the trajectory both acts horizontally, while altitude is held constant. Position data is extracted from PX4's EKF state estimate rather than from Gazebo's overarching physics simulation. While this may introduce absolute estimation errors, the EKF estimate is consistent across all trials, which ensures fair comparative analysis. This is further discussed in section (E) later.

The table of variables used in this experiment is as follows:

Table 2 – Experimental Parameters and Constants

Parameter	Symbol	Value	Unit
Figure-8 Amplitude	A	1.0	m
Angular Frequency	ω	0.3	rad s^{-1}
Flight Altitude	h	2.0	m
Number of Laps	n_{lap}	3	laps
Active Flight Duration	T	64.3	s
Ramp Duration	T_{ramp}	3.0	s
Post-wind-on Settling Time	T_{settle}	20.0	s
Per-trial Timeout	T_{max}	180.0	s
Offboard Setpoint Streaming Rate	f	50	Hz
Wind Magnitude Gust Intensity Factor	W_{maxG}	0.15	~
Wind-direction swing amplitude	θ_{swing}	$\pm$ 15	deg
Wind-direction noise standard deviation	ρ	2	deg

Randomized variables in this experiment are seeded, and drawn from a continuous uniform distribution over their chosen intervals, as depicted in the table below:

Table 3 – Experimental Randomized Variables

Variable	Symbol	Value Range	Unit
Wind Speed	V_w	$[0.02, 5.99]$	m s^{-1}
Wind Direction	θ_w	$[0, 360]$	Deg
Mass Factor	k_m	$[0.80, 1.20]$	~
Inertia Factor	k_I	$[0.80, 1.20]$	~
Gust Direction Swing Period	T_{swing}	$[4, 10]$	s

As seen above, the variables also encode the independent variables that make up the research question, as wind speed, wind direction, and gust direction swing period correspond to gusty wind, while the mass and inertia factors correspond to model mismatch.

To establish a baseline noise floor, one seed (seed number 4, a high-wind case with a wind speed of 5.7 m s^{-1} and a mass factor of 1.19) was flown 5 times, resulting mean RMS was 58.6 ± 0.73 cm. Differences in RMS smaller than this noise floor are not treated as meaningful.

Furthermore, to quantify uncertainty without assuming normally distributed errors, this project used a bootstrap resampling method. By resampling the trials with replacement many times (10,000 resamples) and recomputing the statistic each time, the 2.5th and 97.5th percentiles are taken as the 95% confidence interval (CI). Cis are reported for both the mean RMS per wind bin and the slope of the plot of RMS versus wind speed. Six wind bins are defined, starting from 0 m s^{-1} to 1 m s^{-1}, with each bound incrementing by one m s^{-1} when moving to the next bin.

A total of 200 seeds were evaluated. A failed trial (due to timeout, crash, or connection loss) is recorded with its seed and respective reason; failures are never silently dropped. The batch runner supports resume-on-crash, which allows interrupted overnight runs to skip completed seeds without failure.

(C) Results Analysis

All 200 trials (seeds 3–202) were completed successfully, and no trial was dropped for timeout, crash, or lost connection. The randomized draws exercised the full designed ranges of wind speed (0.02–5.99 m s^{-1}) and mass factor (0.80–1.20).

Firstly, **Table 4** and **Table 5** summarize the bootstrap point estimates and 95% confidence intervals for all reported metrics.

Table 4 – V_w Bootstrap Point Estimates and 95% Confidence Intervals

Mean RMS V_w Bin Range / m s^{-1}	Estimate / cm	95% Confidence Interval / cm
$[0, 1)$	19.6	$[19.1, 20.0]$
$[1, 2)$	24.4	$[23.2, 25.8]$
$[2, 3)$	30.9	$[29.7, 31.9]$
$[3, 4)$	40.7	$[39.3, 42.1]$
$[4, 5)$	50.0	$[48.3, 51.8]$
$[5, 6]$	57.9	$[55.9, 59.9]$

Table 5 – Other Bootstrap Point Estimates and 95% Confidence Intervals

Metric	Estimate	95% Confidence Interval
RMS vs. wind slope / cm s m^{-1}	8.0	$[7.6, 8.4]$
Mass effect / cm (per +10% mass)	0.0	$[-0.6, 0.6]$
Swing-period effect / cm s^{-1}	1.6	$[1.3, 1.9]$

These values are used to explain the graphs **Figure 3** and **Figure 4** in this section.

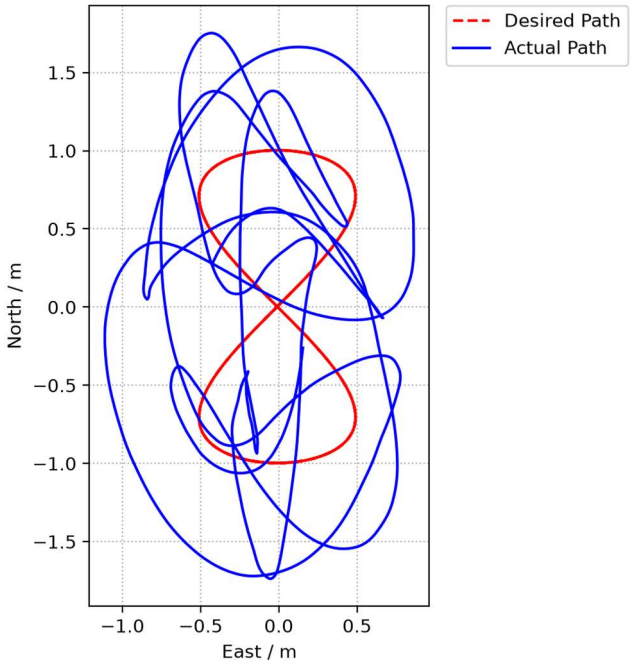


Figure 2 – Example Trajectory Tracking Path (Seed 4)

Figure 2 shows a representative trial (seed 4: wind speed 5.7 m s^{-1}, mass factor 1.19), for which the actual path visibly overshoots the commanded figure-8, giving a final horizontal RMS of 57 cm. This is within roughly two standard deviations of the noise-floor mean of 58.6 ± 0.73 cm measured by repeating seed 4 five times as detailed in section (B), consistent with variability that is attributed to run-to-run gust, rather than any discrepancy between the single-run and batch runs.

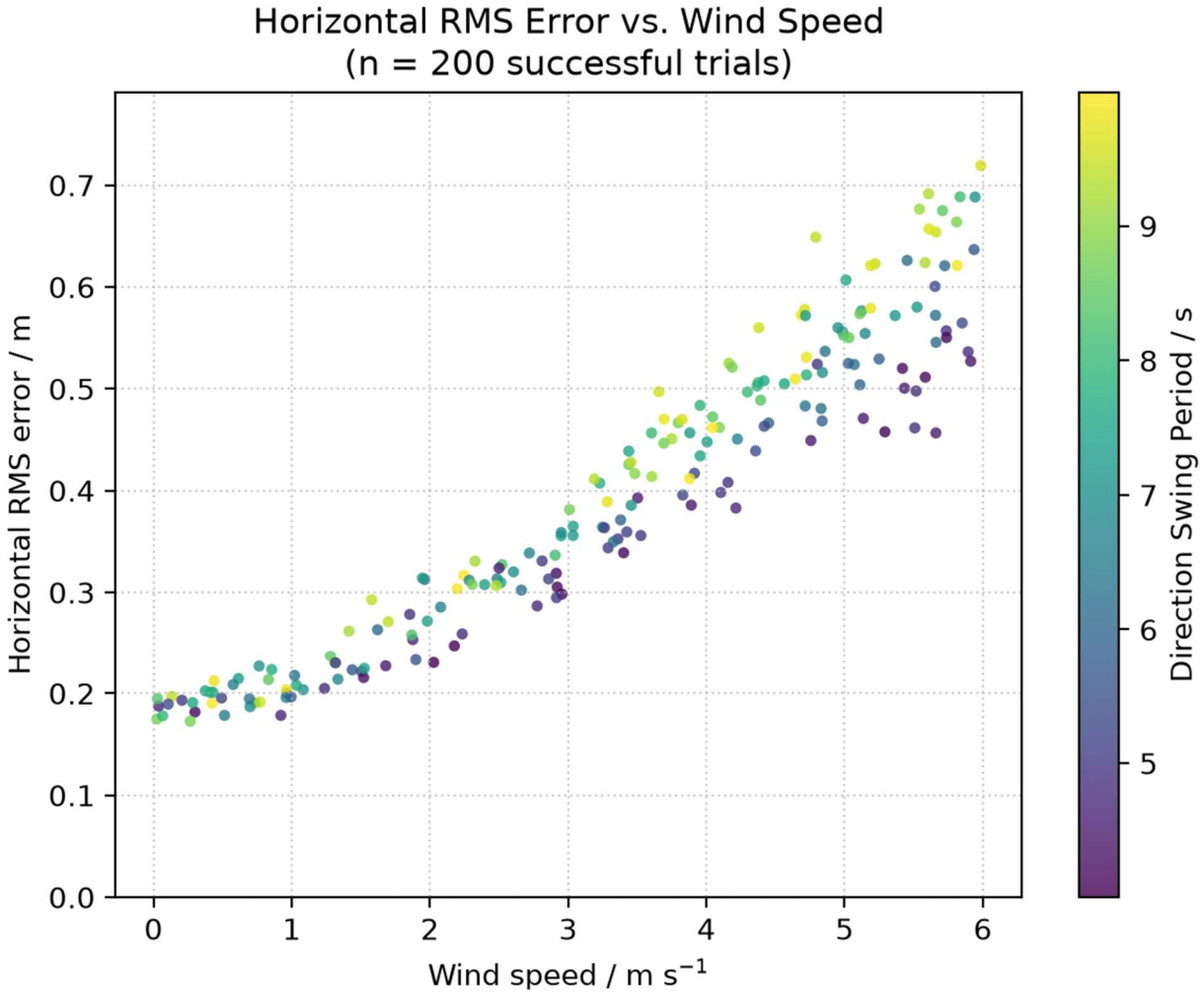


Figure 3 – Horizontal RMS Error vs. Wind Speed

Figure 3 plots the horizontal RMS error σ against wind speed, colored by direction swing period. σ increases with wind speed with a calculated Pearson r coefficient of 0.955. A linear fit of the points gives an estimated slope of 8.0 cm per additional 1.0 m s^{-1} of wind speed. The mean σ increased from 19.6 cm in the calmest wind speed range of [0, 1] m s^{-1}, to 57.9 cm in the windiest wind speed range of [5, 6] m s^{-1}.

The color axis in **Figure 3** further reveals a secondary effect. After the dominant trend arising from wind speed is removed, the residual error is correlated with the sinusoidal direction swing period with a calculated

Pearson r coefficient of 0.67. This is likely because a longer swing period sustains the wind push in a given direction for a longer period before it reverses. Thus, this gives the controller more time to accumulate drift error before the gusting direction reverses again. In contrast, a fast-swinging gust reverses before a large drift error can be reasonably built, which allows for the drift error to be partially averaged out over a lap.

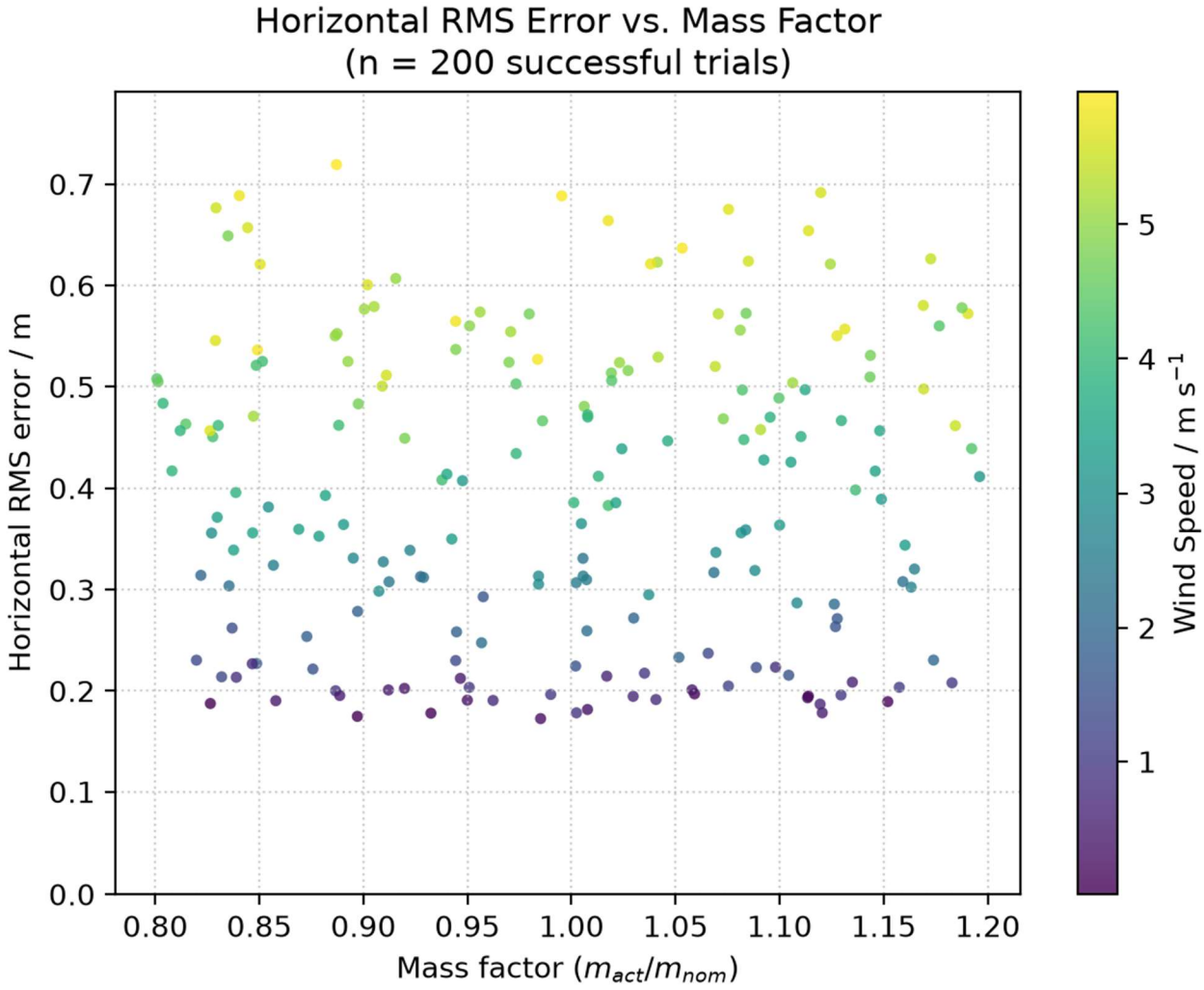


Figure 4 – Horizontal RMS Error vs. Mass Factor

Figure 4 plots the same σ values against the mass factor, colored by wind speed. As seen above, there is no slope that is visible across the [0.80, 1.20] mass factor range tested, and points for a given wind-speed color form a flat band, matching the expected result of **Figure 3**. This is further confirmed numerically: the correlation between σ and mass factor has an r of roughly -0.005, and the bootstrap mass effect is 0.0 cm per +10% mass change, with a 95% CI of [−0.6, 0.6] cm (Table 5). This is an interval comparable in width to the single-trial noise floor itself as detailed in section (B). PX4's stock controller therefore compensates

for mass mismatch over this range with no measurable degradation in horizontal tracking, which juxtaposes clearly the wind and swing-period effects above.

The trends in **Figure 3** are quantified by the bootstrap estimates in **Table 4 and Table 5**; specifically, the wind-speed bins in **Table 4** correspond to successive vertical bands of **Figure 3**. The mean σ rises with every bin, and as the 95% CIs of adjacent bins do not overlap, each increase in wind speed produces a distinguishable increase in error. The bin-to-bin increase is not perfectly uniform, however, ranging from 4.8 cm between the two calmest bins to 9.8 cm between the [2, 3) and [3, 4) m s^{-1} bins; but this reflects the flatter region of **Figure 3** at low wind speeds.

Nevertheless, the linear slope in Table 5 of 8.0 cm per m s^{-1} (95% CI [7.6, 8.4]) remains a good summary of the overall trend. Across the 5 m s^{-1} separating the centers of the calmest and windiest bins, it predicts an increase of 40.0 cm, close to the actual 38.3 cm difference between those same bins in **Table 4**. Likewise, the swing-period effect in **Table 5** of 1.6 cm per additional second (95% CI [1.3, 1.9] cm s^{-1}) quantifies the color gradient in **Figure 3**, corresponding to an average difference in RMS error of approximately 9.6 cm between the fastest and slowest swinging gusts. As this value is an average across all wind speeds, it should also be noted that the color separation in **Figure 3** is visibly narrower at low wind speeds and wider at high wind speeds.

Table 4 and **Table 5** also explain the structure of **Figure 4**. The horizontal color bands in **Figure 4** correspond to the wind-speed bins in Table 4, with the darkest color points present near the 19.6 cm mean of the [0, 1) m s^{-1} bin and the brightest color points centered on the 57.9 cm mean of the [5, 6] m s^{-1} bin, regardless of their position along the mass-factor axis. Furthermore, in **Table 5**, even at either limit of the mass factor's 95% CI (0.6 cm in magnitude per +10% mass), the full [0.80, 1.20] mass factor range would only vary σ by 2.4 cm, compared with the 38.3 cm difference between the calmest and windiest bins in **Table 4.** Thus, the vertical spread in **Figure 4** is due primarily to variation in wind speed, rather than differences in mass factors.

(D) Discussion

Taken together, **Figures 3 and 4**, along with **Tables 4** and **5** answer the question posed in the project overview: PX4's stock cascade controller tracks the figure-8 target trajectory in environments with calm-to-moderate wind speed, but its tracking error grows roughly linearly with wind speed and becomes significant at the higher ends of the wind speed testing ranges. Even at zero wind, the controller carries a baseline horizontal RMS of approximately 19.6 cm in the $[0, 1)$ m s^{-1} wind speed bin; this can be attributed

to the drone's peak acceleration (B.2) on the figure-8 trajectory, rather than any external disturbance. By the $[5, 6]$ m s^{-1} bin, mean RMS has increased to 57.9 cm, with individual trials reaching as high as 72 cm (**Figure 3**). An error of this magnitude is in the same order as the figure-8's own 0.5 m semi-amplitude along its narrower axis, and would likely be unacceptable for close-proximity, precision-based tasks, such as flying near or through structures when wind speed is high.

The insensitivity of σ to mass and inertia mismatch, as seen in **Figure 4**, and **Table 5** (bootstrap estimate of 0.0 cm per gain of 10% of mass, with 95% CI $[-0.6, 0.6]$) can be explained by the nature of PX4's multicopter position controller. Given that the estimator continuously adapts its throttle-to-thrust mapping to the vehicle's actual weight, and that it operates independently of position, velocity, and attitude loops tested in this experiment, a mass change is absorbed before it affects σ. In contrast, wind gusts are external forces that the position loop must directly counteract through feedback alone. This suggests that the stock PX4 tuning already has an adequate margin for the payload or airframe variation as tested in this experiment.

The secondary swing-period effect can also be discussed, as seen in **Table 5** (1.6 cm of residual RMS per additional second of direction swing period, 95% CI $[1.3, 1.9]$). A gust direction that swings quickly pushes the drone one way and then the other before a large error can accumulate, thus rendering the error being partially mitigated during a lap. In contrast, a slow swinging gust behaves more similarly to a slowly varying bias, that the position loop must track continuously, producing a larger deviation. This suggests that, within the tested range, tracking error depends on the timescale of the wind's direction changes as well as on its speed. This cannot be captured by the wind-speed slope metric alone.

(E) Limitations

The primary metric, σ, is computed from PX4's EKF position estimate, rather than Gazebo's simulation ground-truth position (Section (B)). The EKF's own estimation error is therefore a part of every measurement; because the same estimator runs identically in every trial, relative comparisons across wind speed, mass, and swing period remain fair, but the absolute RMS values reported here are not pure measurements of performance in isolation from the errors arisen from state estimation.

Furthermore, the wind itself is a parametric approximation, and not a physical turbulence model. Gazebo's WindEffects plugin drives both magnitude and direction with a fixed sine term, plus a Gaussian noise term (**Table 2**), rather than a continuous spectrum model, such as Dryden or von Kármán turbulence. The swing-period effect identified in Section (C) is therefore tied to this specific sinusoidal parametrization of gust direction; whether a similar frequency-dependent effect would occur under continuous spectrum

atmospheric turbulence models is not established by this experiment. There is also a minor hesitancy when employing the indirect seeding method for per-trial gust creation. As it is conducted through Gazebo's render-engine seed argument rather than a well-known Gazebo documented wind-noise seed, its reproducibility is not necessarily guaranteed.

The Monte Carlo design draws randomized quantities (wind speed, wind direction, mass factor, inertia factor, and direction-swing period) independently and simultaneously from a single pool of 200 trials. This is sufficient to isolate the dominant wind effect and the secondary swing-period effect reported in section (D). However, weaker interactions, such as those between mass and inertia factors, or between wind direction and the figure-8's two unequal axes, are not resolved at this sample size, and wind direction itself was not analyzed in this report despite also being randomized in every trial. Furthermore, the noise floor of 58.6 ± 0.73 cm obtained in section (B) was also established from a single seed (seed 4) repeated five times. This noise floor may not represent run-to-run variability at other wind speeds.

Finally, all results are specific to one vehicle (the x500 quadrotor), one trajectory shape, size, and speed (a 1 m amplitude figure-8 at a ω value of 0.3 rad s^{-1}), and one simulation stack (PX4 SITL with Gazebo Sim 8.15.0). Additionally, the mass and inertia perturbation itself is also simplified. The inertia factor scales the three diagonal inertial terms by one common factor, rather than perturbing them independently along each axes, or introducing off-diagonal products of inertia. Therefore, it represents a simplified approximation of real mass distribution. Moreover, PX4 SITL does indeed approximate real aerodynamic calculations, but does not reproduce them exactly. Some factors, such as propellor-wake interactions and real turbulence are simplified, or absent. Therefore, the qualitative conclusions from above, of strong wind sensitivity, and negligible mass sensitivity are expected to transfer to real hardware, but the specific numerical values obtained in this experiment should not be taken as values that can be attributed to real hardware runs.

(F) Further Expansions

- Implement a wind-feedforward term in the position controller, using PX4's wind estimator, which helps further quantify the wind slope and wind-speed RMS bins relative to the baseline
- Repeat the study with a continuous-spectrum turbulence model (e.g. Dryden or von Kármán, both supported by Gazebo),
- Repeat the noise-floor measurement at several wind speeds, rather than only using seed 4 as a baseline, as the scatter presented in **Figure 3** suggests that a single value is not enough
- Re-run the experiment for other vehicles, vehicle sizes, trajectory shapes, and speeds
- Validate against hardware-in-the-loop or real outdoor flights in light wind conditions to quantify the difference between simulated and actual wind slope or noise floor measurements

References

1. PX4 Autopilot, version v1.18.0-beta1 (development build). Open source autopilot flight control software. https://px4.io / https://github.com/PX4/PX4-Autopilot

2. Gazebo Sim, version 8.15.0. Open Source Robotics Foundation. https://gazebosim.org

3. MAVSDK-Python (mavsdk < 4). MAVLink software development kit for Python. https://github.com/mavlink/MAVSDK-Python

4. Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). Array programming with NumPy. Nature, 585, 357–362.

5. McKinney, W. (2010). Data Structures for Statistical Computing in Python. Proceedings of the 9th Python in Science Conference, 56–61.

6. Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. Computing in Science & Engineering, 9(3), 90–95.

7. Efron, B., & Tibshirani, R. J. (1993). An Introduction to the Bootstrap. Chapman & Hall/CRC.

Appendix

Table 6 – Gazebo WindEffects and Simulation Configuration (Full Listing)

Parameter Path	Value	Notes
force_approximation_scaling_factor	1	Drag coeff. = 1
horizontal.magnitude.time_for_rise	1 s	~
horizontal.magnitude.sin.amplitude_percent	0.15	Gust intensity
horizontal.magnitude.sin.period	surge_period	Surging gusts
horizontal.magnitude.noise.type / mean	Gaussian	~
horizontal.magnitude.noise.stddev	$0.15V_w$	!
horizontal.direction.time_for_rise	1 s	~
horizontal.direction.sin.amplitude	15 degrees	~
horizontal.direction.sin.period	direction_swing_period	~
horizontal.direction.noise.type / mean	Gaussian	~
horizontal.direction.noise.stddev	2 degrees	~
PX4_SYS_AUTOSTART (airframe ID)	4001	Stock x500 tuning
Body mass scaling	mass_new = mass_factor × nominal	~
Body inertia scaling (ixx, iyy, izz)	inertia_new = inertia_factor × nominal	~
Gust/Gazebo RNG seeding mechanism	PX4_GZ_SIM_RENDER_ENGINE='ogre2 --seed <gz_seed>	~
Per-trial hard timeout (RUN_TIMEOUT)	180 s	~